42. Canlı Ortam Gözlemlenebilirliği: Sentry, Datadog ve Log Analizi

Kullanıcılar 'Site Bozuk' Demeden Önce Hataları Canlıda Yakalamak
Bölüm 7 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Bir şirkette canlıdaki bir hatayı ilk olarak müşteriler Twitter'da şikayet edince öğreniyorsanız, gözlemlenebilirlik (Observability) açısından neden sınıfta kalmışsınızdır?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Gözlemlenebilirlik (Observability) 3 temel ayaktan oluşur: Metrikler (Metrics), Loglar (Logs) ve İzler (Traces). - 📌 Sentry, kullanıcı tarayıcısında veya sunucuda patlayan hataları anında yakalayarak tam stack trace ve kullanıcı oturumuyla önünüze serer. - 📌 AI modelleri, binlerce satırlık Datadog/CloudWatch log akışını tarayarak 'anomali' ve 'kök neden' özetini saniyeler içinde çıkarabilir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Hastanenin yoğun bakım ünitesini (ICU) hayal edin.

Hastanın parmağında nabız ölçer (Metrik), başında kalp elektrosu (Trace), hemşirenin elinde saatlik gözlem defteri (Log) vardır. Eğer hastanın kalbi durursa sirenler anında çalar ve doktorlar odaya dalar!

Peki ya yoğun bakımda hiçbir cihaz olmasaydı? Doktorlar hastanın durumunu ancak koridordaki bir ziyaretçi: *"Galiba 3. odadaki hasta nefes almıyor!"* dediğinde öğrenirdi!

İşte canlıya Sentry veya Datadog kurmadan kod atmak, o cihazsız yoğun bakım odasıdır.

Profesyonel bir ekip, kullanıcı hatayı henüz fark etmeden önce Sentry bildirimini alır, düzeltmeyi yapar ve güncellemeyi yayına alır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Gözlemlenebilirliğin (Observability) 3 Sacayağı:

```
┌──────────────────────────────────────────────────┐
│                GÖZLEMLENEBİLİRLİK                │
└──────────────────────┬───────────────────────────┘
                       │
       ┌───────────────┼──────────────┐
       ▼               ▼              ▼
[1. METRİKLER (Metrics)] [2. LOGLAR (Logs)]  [3. İZLER (Traces)]
- "CPU kullanımı %92"   - Yapılandırılmış JSON- "İstek 1500 ms sürdü:
- "Hata oranı %0.05"    - "Kullanıcı giriş yaptı"  1200 ms DB sorgusu,
- Araç: Prometheus      - Araç: Winston / Pino    300 ms Stripe API"
                                                  - Araç: OpenTelemetry
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Observability ve SRE Uzmanı
Next.js / Node.js backend projemize kurumsal düzeyde Sentry entegrasyonu yapacağız.

## GÖREV:
1. `sentry.server.config.ts` yapılandırmasını oluştur (Tracing ve Profiling açık).
2. Hassas kullanıcı verilerinin (şifre, kredi kartı, auth token) Sentry'ye gitmesini
engelleyen `beforeSend` filtresini yaz (Sanitization).
3. Yakalanan bir hataya kullanıcı ID'si ve ek iş bağlamı (tags / extra context)
ekleme örneğini göster.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Kötü: console.error ile loglamak (Sunucu çökünce loglar kaybolur, kimse görmez!)
app.post("/checkout", async (req, res) => {
  try {
    await processPayment(req.body);
  } catch (err) {
    console.error("Hata oldu:", err); // Canlıda hiç kimse fark etmez!
    res.status(500).send("Sunucu hatası");
  }
});
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
import * as Sentry from "@sentry/node";

// 1. Sentry Başlatma ve Güvenlik Filtresi (sentry.server.ts)
Sentry.init({
  dsn: process.env.SENTRY_DSN,
  environment: process.env.NODE_ENV || "production",
  tracesSampleRate: 0.2, // Performans izleme için %20 örnekleme

  // GÜVENLİK: Hassas verileri Sentry'ye göndermeden önce temizle (Scrubbing)
  beforeSend(event) {
    if (event.request?.headers) {
      delete event.request.headers["authorization"];
      delete event.request.headers["cookie"];
    }
    return event;
  }
});

// 2. Güvenli Hata Yakalama ve Bağlam Ekleme
export async function handleCheckout(req: Request, res: Response) {
  try {
    await processPayment(req.body);
    res.json({ success: true });
  } catch (error) {
    // Hatayı zengin bağlamla Sentry'ye bildir
    Sentry.withScope((scope) => {
      scope.setUser({ id: req.user?.id, email: req.user?.email });
      scope.setTag("transaction_type", "subscription_renewal");
      scope.setExtra("cart_payload", req.body);
      Sentry.captureException(error);
    });

    res.status(500).json({ error: "İşlem sırasında beklenmedik bir hata oluştu." });
  }
}
```

💡 **Kod Analizi:** `beforeSend` filtresiyle hassas auth token ve çerezler temizlenerek KVKK / GDPR uyumu sağlanmış; `withScope` ile hatanın tam olarak hangi kullanıcıda ve hangi işlem tipinde patladığı etiketlenmiştir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Loglara şifre, TC kimlik no veya kredi kartı bilgilerini düz metin olarak basmak (PII Leak).
  - **Çözüm:** Hassas alanları log kütüphanelerinde (Pino/Winston) otomatik maskeleyen redactor kullanın.
- **Tuzak:** Sentry'ye her milisaniyede log gönderip kotayı 1 günde bitirmek.

  - **Çözüm:** `tracesSampleRate` oranını production'da makul bir seviyeye (%10 - %20) ayarlayın.
- **Tuzak:** Hata oluştuğunda try/catch ile hatayı sessizce yutup hiçbir yere bildirmemek.
  - **Çözüm:** Catch bloklarında hatayı mutlaka Sentry veya merkezi log sistemine gönderin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 42: Canlı Ortam Gözlemlenebilirliği - Sentry ve Telemetri*

- 🖥️ **Slayt 1:** Gözlemlenebilirliğin (Observability) 3 Sacayağı: Metrics, Logs, Traces.
- 🖥️ **Slayt 2:** Sentry kurulumu ve 'Breadcrumbs' ile hatanın adımlarını izlemek.
- 🖥️ **Slayt 3:** Hassas veri temizliği (PII Data Scrubbing) ile KVKK uyumu.
- 🖥️ **Slayt 4:** Bölüm 7 Özeti: Hata Ayıklama, Profiling ve Sistem Sağlığı.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Sentry dashboard'unu açın. Canlı API'de kasıtlı bir hata tetikleyin. Sentry'nin 2 saniye içinde hatayı nasıl yakaladığını, kullanıcının tıkladığı önceki butonları (breadcrumbs) ve stack trace'i harita gibi gösterdiğini inceleyin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Kişisel bir projenize ücretsiz bir Sentry hesabı bağlayın (`@sentry/node` veya `@sentry/react`) ve canlıda kasıtlı bir hata yakalatarak dashboard'da inceleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Yazılım sistemlerinde 'Gözlemlenebilirlik' (Observability) hangi 3 temel veriden oluşur? - A) HTML, CSS, JavaScript - B) Metrikler (Metrics), Loglar (Logs) ve Dağıtık İzler (Traces) - C) Klavye, Fare, Ekran - D) Git, Docker, Kubernetes > **Doğru Yanıt:** B > **Açıklama:** Modern gözlemlenebilirlik; sayısal metrikler, olay logları ve isteklerin tüm sistem boyunca izlediği dağıtık izlerden (traces) meydana gelir.

Soru 2: Sentry gibi hata izleme araçlarında 'Breadcrumbs' (Ekmek Kırıntıları) ne işe yarar? - A) Kuşlara yem vermek için - B) Hata patlamadan önceki son 10 adımda kullanıcının hangi butonlara bastığını, hangi URL'leri ziyaret ettiğini ve hangi ağ çağrılarının yapıldığını adım adım göstermek - C) Şifreleri kaydetmek - D) Dosyaları sıkıştırmak > **Doğru Yanıt:** B > **Açıklama:** Breadcrumbs, bir çöküş anından önce sistemde nelerin yaşandığını kronolojik olarak göstererek hatayı dakikalar içinde yeniden üretmeyi sağlar.